MedShare

Medicine-inventory coordination platform

Complete project guide: what it does, how it is built, how to run it, the database, the demo accounts, testing and performance.

Item	Value
Project type	University DBMS project (database design + full-stack application)
Stack	PostgreSQL 16 · Node.js / Express / TypeScript · React / TypeScript
Repository	C:\Users\harsha\MedShare (git branch `phase-4/integration`)
Status	Phases 1–6 complete; all test suites passing
Data	Entirely synthetic. No real institutions, stock or patients.

Contents

1. What MedShare is 4

1.1 The problem it solves . . . 4

1.2 Objectives . . . 4

1.3 How the project was built . . . 4

2. Technology and architecture 5

2.1 Technology stack . . . 5

2.2 Architecture . . . 5

2.3 Ports and URLs . . . 5

3. How to run the project 6

3.1 Prerequisites . . . 6

3.2 First-time setup (already done on this machine) . . . 6

3.3 Every time you want to use it . . . 6

3.4 Useful commands . . . 6

4. Demo accounts 7

4.1 Platform and organisation accounts . . . 7

4.2 Facility accounts . . . 7

4.3 What each role can do . . . 8

4.4 Good accounts for a quick tour . . . 8

5. The database 9

5.1 Engine and connection details . . . 9

5.2 Tables (35) . . . 9

5.3 Key relationships (ER model) . . . 9

5.4 Migrations (schema as code) . . . 10

5.5 Integrity features . . . 11

5.6 Synthetic seed data . . . 11

6. How the main features work 12

6.1 Inventory and the ledger . . . 12

6.2 Transfer lifecycle . . . 12

6.3 Shortages, expiry and demand . . . 12

6.4 Redistribution engine . . . 12

6.5 Stock requests . . . 12

7. Transactions and concurrency 13

7.1 The 150-unit example . . . 13

7.2 Other protections . . . 13

8. The REST API 13

9. The web application 14

10. Testing and verification 17

10.1 Browser verification (end to end) 17

10.2 Bugs found and fixed in Phase 5 17

11. Performance study 17

12. Project files and documentation 17

13. Limitations 18

1. What MedShare is

MedShare is a platform that helps healthcare facilities in a region share medicine stock. It shows, across the whole network, which facilities are **running short** of a medicine, which hold **excess** stock, and which batches are **close to expiry**. It then helps staff move stock from where it is not needed to where it is, through an approval-controlled **request → approve → dispatch → receive** transfer workflow.

1.1 The problem it solves

Facilities in the same district often hold the wrong stock at the same time. One hospital runs out of an essential antibiotic while a pharmacy 20 km away holds surplus that will expire next month. The information sits in separate stock registers, so nobody sees the match until it is too late: the patient goes without, and the surplus is thrown away.

1.2 Objectives

- **One consistent view of stock**: batch-level inventory per facility with *available*, *reserved* and *damaged* quantities that can never go negative or disagree with their history.
- **Early warning**: automatic shortage alerts (days of stock left, from real or estimated demand) and expiry alerts (7, 30 and 60 days).
- **Redistribution recommendations** that pair shortages with surplus and near-expiry stock, taking demand, safety stock, first-expiry-first-out (FEFO), distance, dispatch radius and cold chain into account. They are **advisory only**.
- **A safe transfer workflow**: stock reserved atomically at approval, no double allocation even under simultaneous requests, no self-approval, an extra organisation approval for transfers between organisations, and safe retries.
- **Accountability**: an append-only movement ledger and audit log, least-privilege database roles, per-facility access.
- **Demonstrate DBMS concepts**: normalised design (3NF/BCNF), constraints, triggers, stored procedures, views, indexes, transactions and isolation, and analytical SQL.

Simulated platform. MedShare is not connected to any real hospital, pharmacy or government system. Every organisation, facility, user, manufacturer and number in it is synthetic and labelled "(Synthetic)". Real-world use would need institutional authorisation and regulatory validation.

1.3 How the project was built

Phase	What was delivered
1. Design	Requirements, ER model, relational schema, functional dependencies, normalisation, concurrency and engine design
2. Database	PostgreSQL schema, constraints, triggers, procedures, views, roles, synthetic seed, 99 database tests
3. API	Express + TypeScript REST API with JWT authentication, permission checks, validation, API tests
4. Frontend integration	React app connected to the real API, new pages, verified end-to-end in a browser
5. Testing and optimisation	Three bugs fixed with regression tests; EXPLAIN ANALYZE study on 2 million rows; migrations 008 and 009
6. Documentation	README, DBMS documentation, ER diagram, 24-query catalogue, demonstration walkthrough

2. Technology and architecture

2.1 Technology stack

Layer	Technology	Why
Database	PostgreSQL 16 in Docker	Procedures, triggers, exclusion constraints, trigram/BRIN indexes, window functions, row locking
Backend	Node.js 20+, Express, TypeScript, Zod, JWT, bcrypt, node-postgres, Prisma (typed client)	Thin, typed API; business integrity stays in the database
Frontend	React 18, TypeScript, Vite, Tailwind CSS 4, TanStack Query, React Router, React Hook Form, Recharts	Fast, accessible UI with cached server state and charts
Tests	SQL test scripts, multi-connection concurrency tests, Vitest + Supertest, Testing Library	Tests run against a real PostgreSQL, not mocks

2.2 Architecture

Tier	Responsibility
Browser (React)	Presentation only. Shows or hides buttons based on the user's permissions, but never decides stock or permissions itself.
API (Express)	Verifies the JWT, validates every request with Zod, checks the user's facility scope, passes the Idempotency-Key, maps database errors to HTTP codes. Connects as the restricted role `medshare_app`.
Database (PostgreSQL)	The system of record and the enforcement point: constraints, the ledger trigger, the transfer state machine, SECURITY DEFINER procedures that re-check permissions, views for reporting.

Three design rules make the database the authority:

- Stock balances change **only** through the `inventory_movements` ledger. A trigger applies each movement to the balance; nothing else may update it, not even a superuser.
- The API's database role **cannot** write inventory, the ledger or transfers directly. It must call procedures that check the user's permission inside the database.
- A reconciliation function proves at any time that every balance equals the sum of its ledger movements (result: 0 discrepancies).

2.3 Ports and URLs

Service	Address	Started by
Frontend (web app)	http://localhost:5173	`npm run dev` in `frontend/`
API	http://localhost:4000/api/v1 (health: /api/v1/health)	`npm run dev` in `backend/`
PostgreSQL	localhost:5433 (container port 5432)	`npm run db:up` (Docker container `medshare-postgres`)

3. How to run the project

3.1 Prerequisites

- **Docker Desktop**, started and running (it hosts the PostgreSQL database).
- **Node.js 20 or newer** (installed: v22.19.0).
- The repository at `C:\Users\harsha\MedShare` with a `.env` file in its root (already created).

3.2 First-time setup (already done on this machine)

```
cd C:/Users/harsha/MedShare
copy .env.example .env        (then change the passwords and JWT_SECRET)
cd backend
npm install
npm run db:up                  starts PostgreSQL 16 on localhost:5433
npm run db:reset               migrations 000-009 + synthetic seed (~20 s)
cd ../frontend
npm install
```

3.3 Every time you want to use it

Terminal 1, the database and the API:

```
cd C:/Users/harsha/MedShare/backend
npm run db:up
npm run dev
```

Terminal 2, the web app:

```
cd C:/Users/harsha/MedShare/frontend
npm run dev
```

Then open **http://localhost:5173** and sign in with one of the demo accounts in section 4. In development builds, the sign-in page shows the main demo accounts as one-click buttons.

3.4 Useful commands

Command (run in backend/ unless stated)	What it does
`npm run db:reset`	Wipes the database and reloads the clean synthetic seed. Do this before a demo.
`npm run db:migrate`	Applies any new migrations (refuses edited ones; they are checksummed).
`npm run db:test`	77 SQL integrity checks + 22 multi-connection concurrency checks.
`npm test`	55 API integration tests (creates and deletes its own test database).
`npm run db:queries`	Runs the 24 documented SQL queries, read-only, as the application role.
`npm run perf:build` / `perf:explain`	Builds the 2-million-row performance database and runs the EXPLAIN study.
`npm test` in frontend/	19 frontend unit and component tests.
`npm run build` in frontend/	Type-check and production build.

To stop: press **Ctrl+C** in each terminal. The database container keeps running in Docker until you stop it with `docker compose stop` in the repository root; its data is kept between restarts.

4. Demo accounts

Every demo account uses the same password: MedShare@2026
These accounts are synthetic and created by the seed (`npm run db:reset` recreates them). They exist only in your local database.

4.1 Platform and organisation accounts

Email (username)	Name	Role and scope
admin@medshare.health	Asha Raman	System administrator: whole network, users, batches, audit. Cannot approve transfers (segregation of duties).
auditor@medshare.health	Farhan Siddiqui	Platform auditor: read-only, whole network, audit log.
org1.admin@medshare.health	Pooja Kulkarni	Organisation admin: Deccan Public Health Network (FAC-001 to FAC-005).
org2.admin@medshare.health	Imran Bhat	Organisation admin: Kaveri Hospitals Trust (FAC-006 to FAC-008).
org3.admin@medshare.health	Shruti Devi	Organisation admin: Namma Pharmacy Cooperative (FAC-009 to FAC-011).
org4.admin@medshare.health	Tejas Kumar	Organisation admin: Southern Medical Supplies Depot (FAC-012, FAC-013).
org5.admin@medshare.health	Asha Nair	Organisation admin: Arogya Community Health Foundation (FAC-014, FAC-015).
org2.auditor@medshare.health	Bhavana Acharya	Auditor scoped to Kaveri Hospitals Trust only.

All display names end in "(Synthetic)" in the app.

4.2 Facility accounts

Each facility has a **facility manager** (`fac-NNN.manager@medshare.health`) and an **inventory manager** (`fac-NNN.inventory@medshare.health`).

Code	Facility	Facility manager login	Inventory manager login
FAC-001	Bengaluru Central District Hospital Hospital · Deccan	`fac-001.manager@medshare.health` Kavya Menon	`fac-001.inventory@medshare.health` Naveen Shetty
FAC-002	Yelahanka Community Health Centre CHC · Deccan	`fac-002.manager@medshare.health` Rekha Prasad	`fac-002.inventory@medshare.health` Mohan Kulkarni
FAC-003	Hoskote Primary Health Centre PHC · Deccan	`fac-003.manager@medshare.health` Mohan Raman	`fac-003.inventory@medshare.health` Asha Hegde
FAC-004	Ramanagara District Hospital Hospital · Deccan	`fac-004.manager@medshare.health` Kavya Gowda	`fac-004.inventory@medshare.health` Naveen Nair
FAC-005	Magadi Primary Health Centre PHC · Deccan	`fac-005.manager@medshare.health` Rekha Iyer	`fac-005.inventory@medshare.health` Mohan Khan
FAC-006	Kaveri General Hospital Jayanagar Hospital · Kaveri	`fac-006.manager@medshare.health` Asha Murthy	`fac-006.inventory@medshare.health` Kiran Joshi
FAC-007	Kaveri Hospital Mysuru Hospital · Kaveri	`fac-007.manager@medshare.health` Naveen Prasad	`fac-007.inventory@medshare.health` Rekha Kulkarni
FAC-008	Kaveri Clinic Whitefield CHC · Kaveri	`fac-008.manager@medshare.health` Arjun Kumar	`fac-008.inventory@medshare.health` Kavya Reddy
FAC-009	Namma Pharmacy Malleshwaram Pharmacy · Namma	`fac-009.manager@medshare.health` Kiran Rao	`fac-009.inventory@medshare.health` Arjun Acharya
FAC-010	Namma Pharmacy Tumakuru Pharmacy · Namma	`fac-010.manager@medshare.health` Asha Rao	`fac-010.inventory@medshare.health` Kiran Acharya
FAC-011	Namma Pharmacy Electronic City Pharmacy · Namma	`fac-011.manager@medshare.health` Naveen Bhat	`fac-011.inventory@medshare.health` Rekha Devi
FAC-012	Southern Depot Peenya Warehouse · Southern	`fac-012.manager@medshare.health` Mohan Iyer	`fac-012.inventory@medshare.health` Asha Khan

Code	Facility	Facility manager login	Inventory manager login
FAC-013	Southern Depot Mysuru Warehouse · Southern	`fac-013.manager@medshare.health` Arjun Menon	`fac-013.inventory@medshare.health` Kavya Shetty
FAC-014	Arogya CHC Chikkaballapur CHC · Arogya	`fac-014.manager@medshare.health` Mohan Murthy	`fac-014.inventory@medshare.health` Asha Joshi
FAC-015	Arogya PHC Devanahalli PHC · Arogya	`fac-015.manager@medshare.health` Kiran Sheikh	`fac-015.inventory@medshare.health` Arjun Patil

Org: Deccan = Deccan Public Health Network, Kaveri = Kaveri Hospitals Trust, Namma = Namma Pharmacy Cooperative, Southern = Southern Medical Supplies Depot, Arogya = Arogya Community Health Foundation. Password for all: MedShare@2026.

4.3 What each role can do

Role	Permissions
System administrator	Manage users, organisations, facilities, catalogue; quarantine/recall batches; run the recommendation engine; read audit and reports. No stock writes, no transfer approval.
Organisation administrator	Manage its organisation's users and facilities; organisation-stage approval of cross-organisation transfers; run the engine; reports and audit for its facilities.
Facility manager	Stock operations, stock-policy settings, stock requests, request/approve/dispatch/receive transfers at its facility.
Inventory manager	Receipts, dispensing, damage, write-offs, stock requests, request/dispatch/receive transfers. Cannot approve transfers.
Auditor	Read-only access to stock, reports and the audit log (platform-wide or one organisation).

4.4 Good accounts for a quick tour

- **fac-002.inventory@medshare.health**: sees shortages at Yelahanka CHC and requests stock.
- **fac-001.manager@medshare.health**: approves and dispatches that request from Bengaluru Central.
- **admin@medshare.health**: network-wide dashboard, analytics, users, batches and audit log.

5. The database

5.1 Engine and connection details

Setting	Value
Database engine	PostgreSQL 16 (image `postgres:16-alpine`)
Docker container	`medshare-postgres` (compose project `medshare`, volume `medshare-pgdata`)
Host and port	localhost : 5433 (from your PC); 5432 inside the container
Main database	`medshare` (application data, ~19 MB)
Other databases	`medshare_perf`: 2-million-row performance-test copy (~642 MB, kept on request). `medshare_api_test` is created and deleted by `npm test`.
Superuser (migrations, seed, tests)	`medshare_admin`, password = `POSTGRES_ADMIN_PASSWORD` in `.env`
Application role (used by the API)	`medshare_app`, password = `MEDSHARE_APP_PASSWORD` in `.env`
Owner role (owns all objects)	`medshare_owner` (no login; migrations run as it)
Read-only role	`medshare_readonly` (views only)
Timezone	Asia/Kolkata

The two database passwords were generated randomly when `.env` was created and are kept only in `C:\Users\harsha\MedShare\.env` (git-ignored), so this document can be shared without exposing them. Connect from a terminal without a password using:

```
docker exec -it medshare-postgres psql -U medshare_app -d medshare       (as the API role)
docker exec -it medshare-postgres psql -U medshare_admin -d medshare     (as superuser)
```

5.2 Tables (35)

Group	Tables
Identity and access	users, roles, permissions, role_permissions, user_roles, facility_users
Network	organizations, organization_agreements, regions, facilities
Catalogue	medicine_categories, medicine_units, substances, medicines, medicine_ingredients, manufacturers, medicine_batches
Stock	inventory, movement_types, inventory_movements, facility_medicine_settings, engine_config
Demand and redistribution	stock_requests, stock_request_items, redistribution_recommendations
Transfers	transfer_requests, transfer_items, stock_request_fulfilments, transfer_approvals, transfer_status_history
Monitoring and accountability	expiry_alerts, shortage_alerts, notifications, audit_logs, idempotency_keys

5.3 Key relationships (ER model)

- An **organisation** operates many **facilities**; facilities sit in a recursive **region** tree (state › district › taluk).
- A **medicine** is made of one or more **substances** (`medicine_ingredients`, with strength), so combination products are supported.
- A **batch** is one physical lot from a manufacturer. `inventory` links facility × batch and stores the three balances.
- Every stock change is a row in `inventory_movements` (the ledger); `movement_types` says how each type changes the balances.
- A **transfer** has lines (`transfer_items`), one approval per required stage, and a full status history.
- A **stock request** line can be fulfilled by several transfer lines (`stock_request_fulfilments`).
- Two organisations may exchange stock only while an **agreement** is in force (no overlapping periods, enforced by an exclusion constraint).

The full diagram is in `ER_DIAGRAM.md`; the data dictionary, functional dependencies and normalisation proof (1NF → BCNF) are in `backend/docs/PHASE1_DATABASE_DESIGN.md`.

5.4 Migrations (schema as code)

File	Contents
000_bootstrap	Database roles, schema ownership, timezone
001_schema	Types, 35 tables, keys, CHECK/UNIQUE/exclusion constraints, indexes, comments (data dictionary)
002_core	Event clock, RBAC data and permission function, partner authorisation, distance, idempotency and audit helpers
003_triggers	Ledger → balance, state machine, alerts, audit, append-only and integrity triggers
004_procedures	Transfer workflow, stock operations, stock requests, reconciliation, scheduled jobs, redistribution engine
005_views	Recursive trees, inventory detail, demand, shortage status, transfer timeline, analytics, materialized monthly consumption
006_privileges	Least-privilege grants
007_trigger_security	Trigger functions run as SECURITY DEFINER
008_idempotent_stock_actions	Exactly-once dispense, damage, dispose, write-off and adjust (Phase 5 fix)
009_query_performance	fn_today() made STABLE and two views rewritten (Phase 5 speed-up)

Migrations are checksummed: editing one that has already been applied is refused, so changes always go in a new file.

5.5 Integrity features

Feature	Examples in MedShare
CHECK constraints	Quantities ≥ 0; expiry after manufacture; minimum ≤ reorder level; received + damaged = dispatched; source ≠ destination; code formats
UNIQUE / partial unique	Batch lot per manufacturer; one inventory row per facility × batch; one open alert per facility × medicine
Foreign keys	ON DELETE RESTRICT almost everywhere, so history can never disappear through a cascade
Exclusion constraint	No two overlapping agreement periods for the same pair of organisations
Triggers (33)	Ledger applies movements to balances; inventory cannot be edited directly; ledger, audit and history are append-only; transfer state machine; shortage alerts open/escalate/resolve automatically
Stored procedures (8) and functions (55)	Approve (reserves stock atomically), reject, cancel (releases stock), dispatch, receive, goods receipt, stock actions, reconciliation, expiry/shortage jobs, recommendation engine
Views (14) + 1 materialized view	Inventory detail with expiry buckets, demand, shortage status, transfer timeline (LAG/LEAD), analytics, monthly consumption
Indexes (~100)	Composite (FEFO, queues), partial (in-stock rows), covering, GIN trigram (search), BRIN (time-ordered ledger and audit)

5.6 Synthetic seed data

Entity	Seeded
Organisations / facilities / users	5 / 15 / 38
Regions	18 (Karnataka › 6 districts › 11 taluks)
Medicines / batches	50 (7 combinations, 5 cold-chain) / 150 from 6 manufacturers
Inventory rows / ledger movements	≈ 1,840 / ≈ 9,800 (six months of activity)
Stock policies	634 (≈ 100 using estimated demand)
Transfers / stock requests	42 (received, rejected, cancelled, in progress) / 22
Agreements	6 (one expired, to demonstrate the rule)

The seed is created by calling the real procedures in date order, so balances, alerts and analytics are internally consistent; it ends by asserting that reconciliation finds 0 discrepancies. Dates are relative to "today", so expiry windows always look current.

6. How the main features work

6.1 Inventory and the ledger

Each inventory row (one batch at one facility) has three buckets: **available**, **reserved** (promised to an approved transfer) and **damaged**. Every change is a ledger movement whose effect on the buckets is defined once in `movement_types`:

Movement	Effect	Movement	Effect
RECEIPT	+ available	DAMAGE	− available, + damaged
DISPENSE	− available	DISPOSAL	− damaged
RESERVE	− available, + reserved	EXPIRY_WRITE_OFF	− available
RELEASE	+ available, − reserved	ADJUSTMENT_IN / OUT	± available
TRANSFER_OUT	− reserved	TRANSFER_IN	+ available
TRANSIT_DAMAGE	+ damaged (at destination)		

Expired is never stored as a status: it is derived from the expiry date, so it can never be out of date. FEFO (first expiry, first out) is used everywhere stock is picked.

6.2 Transfer lifecycle

Step	Who	What the database does
Request	Staff with transfer:request at the receiving (pull) or sending (push) facility	Validates route, agreement, distance, cold chain and batch eligibility. Creates a PENDING transfer. **No stock is reserved.**
Approve	Source facility manager (+ source organisation admin if cross-organisation)	Records the stage. When every stage is approved it locks the stock rows and reserves all lines atomically, or fails and stays PENDING.
Dispatch	Source staff with transfer:dispatch	Refuses recalled, quarantined or now-expired batches. Moves reserved units out (TRANSFER_OUT).
Receive	Destination staff with transfer:receive	Adds received units (TRANSFER_IN) and damaged units (TRANSIT_DAMAGE). Closes fully received stock requests.
Reject / Cancel	Approver / requester or managers	Reject: PENDING only, reason required. Cancel: PENDING or APPROVED; releases any reservation.

Rules enforced in the database: a requester cannot approve their own transfer; a system administrator cannot approve; every status change is written to the history with the acting user; illegal transitions are rejected.

6.3 Shortages, expiry and demand

- **Days of cover** = usable stock ÷ daily demand. Below the minimum level is **CRITICAL**; below the reorder level is **LOW**.
- **Demand basis**: *Measured* (from the last 30 days of dispensing), *Estimated (simulated)* or *Estimated (configured)*. If a facility dispensed nothing because it was out of stock, measured demand would wrongly read zero, so the estimate is used and the row is flagged *Possibly censored*.
- **Expiry alerts** are raised for stock that is expired or expires within 7, 30 or 60 days; expired stock can only be written off.

6.4 Redistribution engine

The engine runs on one consistent snapshot (REPEATABLE READ). First it covers shortages, using the earliest-expiring eligible stock from the nearest facilities; then it looks for near-expiry surplus that another facility will use in time. Every recommendation has a priority score and a plain-language explanation. **Accepting a recommendation only creates a PENDING transfer**; people still approve it.

6.5 Stock requests

A facility records what it needs (priority, needed-by date, lines). Transfer lines can be linked to request lines; the app shows received, in-progress and outstanding quantities, and the request closes automatically when every line has been received.

7. Transactions and concurrency

Writes run at **READ COMMITTED with explicit row locks** (`SELECT … FOR UPDATE`), which gives a predictable "wait, then re-check". Analytics and the recommendation engine run at **REPEATABLE READ** so every number comes from one snapshot. SERIALIZABLE is demonstrated in the tests, including the automatic retry.

7.1 The 150-unit example

Facility A has 150 units. Transfers to B and C, for 100 units each, are approved at the same moment.

#	What happens
1	Both approvals call the reservation procedure, which locks A's inventory row with FOR UPDATE.
2	Approval 1 gets the lock, sees 150 ≥ 100, reserves 100 and commits.
3	Approval 2 was waiting on the lock. It re-reads the committed row (50 available), raises INSUFFICIENT_STOCK and rolls back completely. Its transfer stays PENDING.
4	Final state: 50 available, 100 reserved. Never more than 150 promised. A CHECK constraint (no negative stock) is the last line of defence.

7.2 Other protections

Risk	Protection
Double allocation / lost update	Row locks; balances changed only by the ledger trigger
Negative stock	Re-check under the lock, then CHECK constraints
Expired or recalled stock sent	Eligibility re-checked at approval and again at dispatch
Deadlocks	Rows locked in id order; the API retries deadlock (40P01) and serialisation (40001) errors
Duplicate request on retry	Idempotency-Key recorded in the same transaction; same key + different body is refused
Half-finished operation	Each procedure is one transaction: all lines are reserved, or none

The concurrency test suite proves this with real parallel connections, e.g. 20 simultaneous approvals of 10 units against 100 units: exactly 10 succeed and 10 fail, and reconciliation reports 0 discrepancies afterwards.

8. The REST API

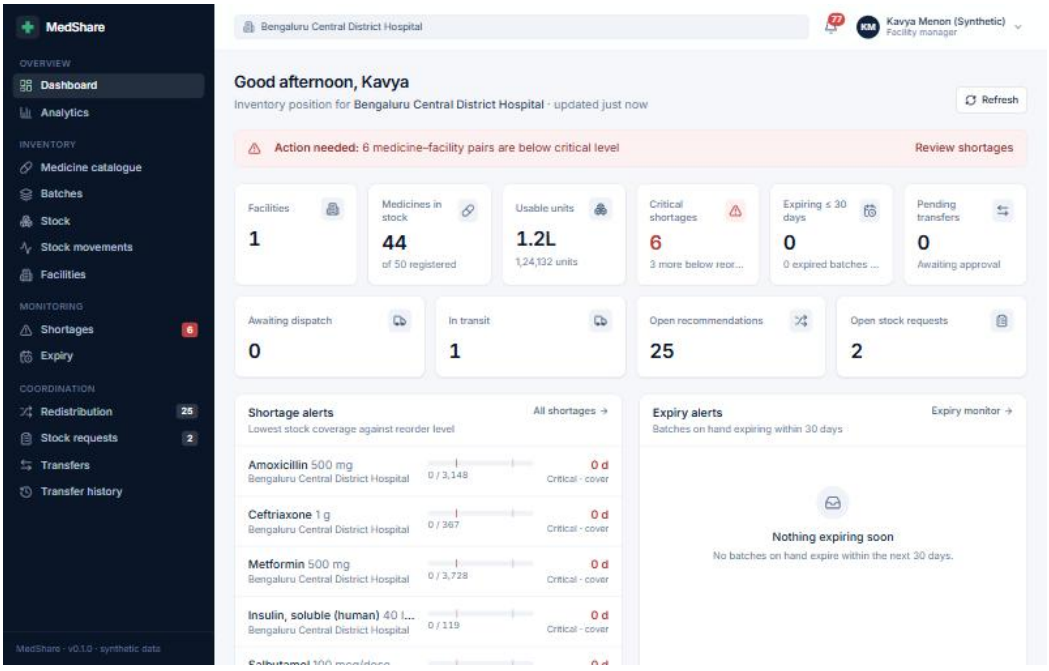
Base URL `/api/v1`; JSON with camelCase keys; integer ids; Bearer-token authentication on every endpoint except login and health; lists return `{ data, meta }` with page, pageSize, total and totalPages; errors return `{ error: { code, message, details, requestId } }`. The complete reference is `API_CONTRACT.md`.

Area	Main endpoints
Auth	POST /auth/login, GET /auth/me, POST /auth/change-password (no self-registration)
Users	GET/POST /users, PATCH /users/:id, grant/revoke organisation and facility roles
Network	GET /regions, /organizations, /facilities (+ /:id, /summary, /settings), PUT stock policy
Catalogue	GET/POST /medicines, GET /medicines/:id, /medicines/:id/sources, /catalogue/*, /batches, POST /batches/:id/status
Inventory	GET /inventory, /inventory/movements, POST /inventory/receipts, /inventory/:id/{dispense, damage, dispose, write-off, adjust}
Alerts	GET /alerts/shortages, /alerts/expiry, /alerts/expiry/summary, POST acknowledge; GET /notifications
Redistribution	GET /redistribution/recommendations, POST /run, /:id/accept, /:id/dismiss
Transfers	GET/POST /transfers, GET /transfers/:id, POST /:id/{approve, reject, cancel, dispatch, receive}
Stock requests	GET/POST /stock-requests, POST /:id/cancel

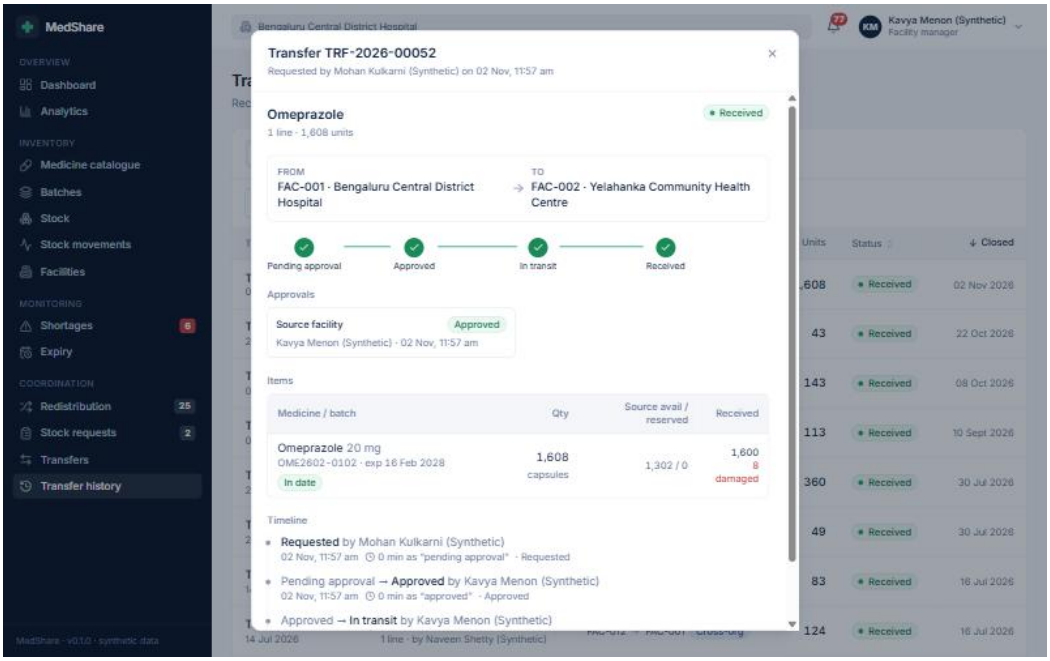
Area	Main endpoints
Dashboard and analytics	GET /dashboard/summary, GET /analytics/overview
Audit and admin	GET /audit-logs, POST /admin/reconciliation, /admin/jobs/:job, GET /admin/roles

9. The web application

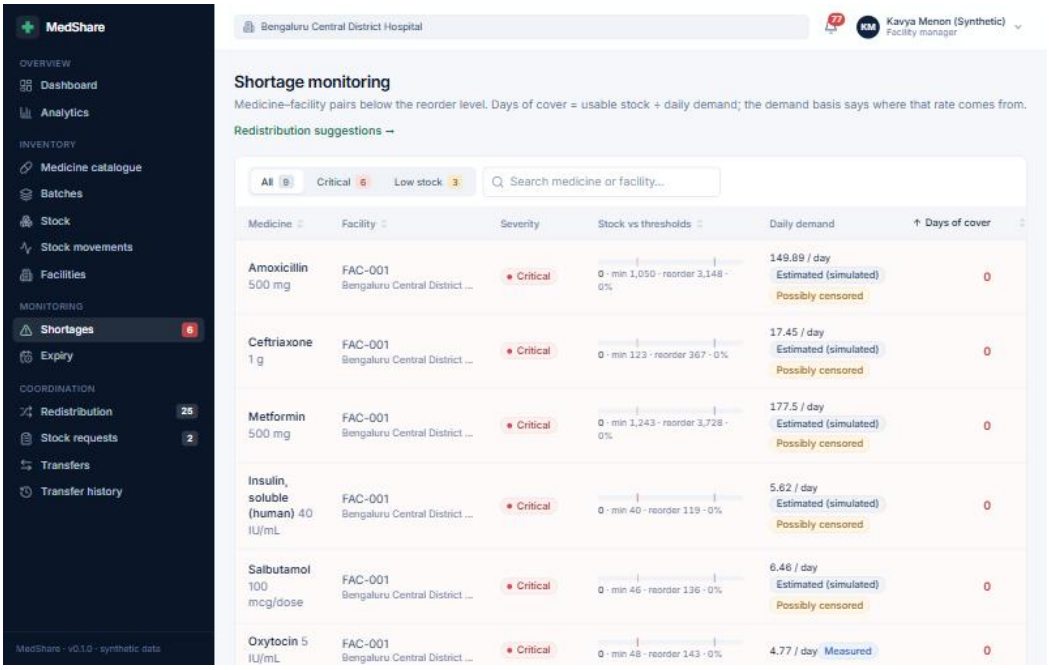
Page	What it shows / does
Dashboard	Action banner, KPI tiles linking to filtered views, shortage and expiry panels, inventory distribution, recent transfers and movements
Analytics	Distribution, availability by region (recursive roll-up), expiry outlook, shortage risk, recurring shortages, wastage, transfer volume and lead time, top consumed with month-over-month change; every chart has a table view
Medicine catalogue	Search by name, code, substance or manufacturer; filters; add-medicine form with an ingredients editor; detail page with stock by facility and batches
Batches	Network lots, holdings, quarantine/recall (system admin)
Stock	Inventory rows with available/reserved/damaged; receive stock; dispense, damage, dispose, write-off, adjust; stock history; send to another facility
Stock movements	The full ledger with signed deltas, filters by type, facility and date
Facilities	Directory; detail with KPIs, shortages and editable stock policy for facilities in your scope
Shortages / Expiry	Critical and low tabs with demand basis; expiry windows; acknowledge, source stock, redistribute, write off
Redistribution	Prioritised, explained recommendations; accept (creates a pending transfer) or dismiss; "Run engine" for admins
Stock requests	Create, cancel, track received / in-progress / outstanding, source a line
Transfers / history	"Needs my action" view; multi-item request dialog; detail with stages, approvals, items and timeline; receive with damaged quantities
Users / Audit log	User and role management; append-only audit entries with before/after values



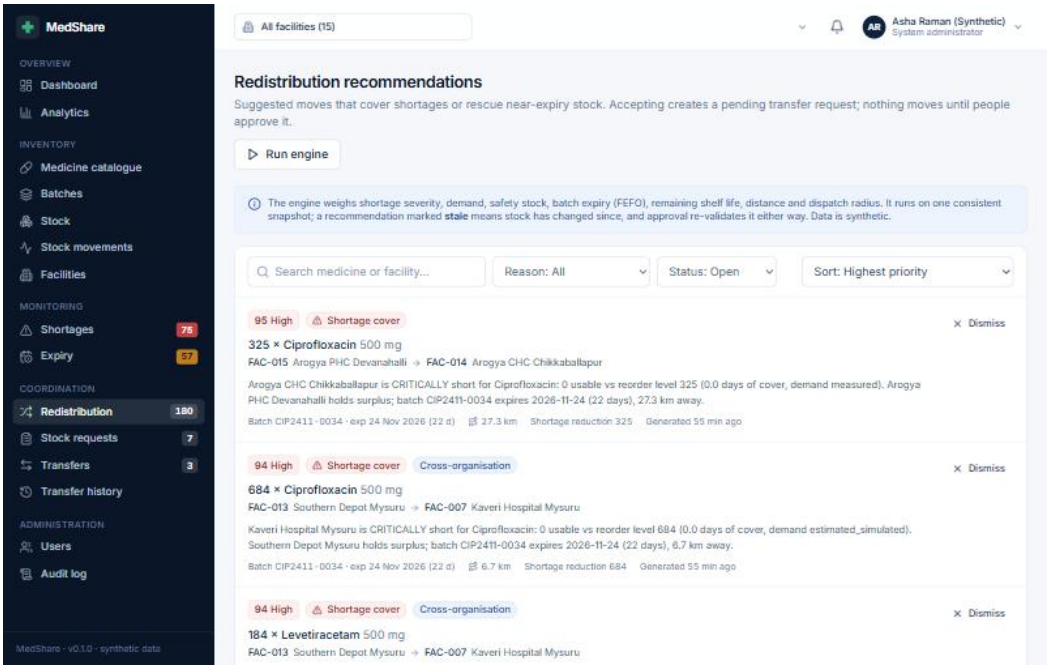
Dashboard for the FAC-001 facility manager



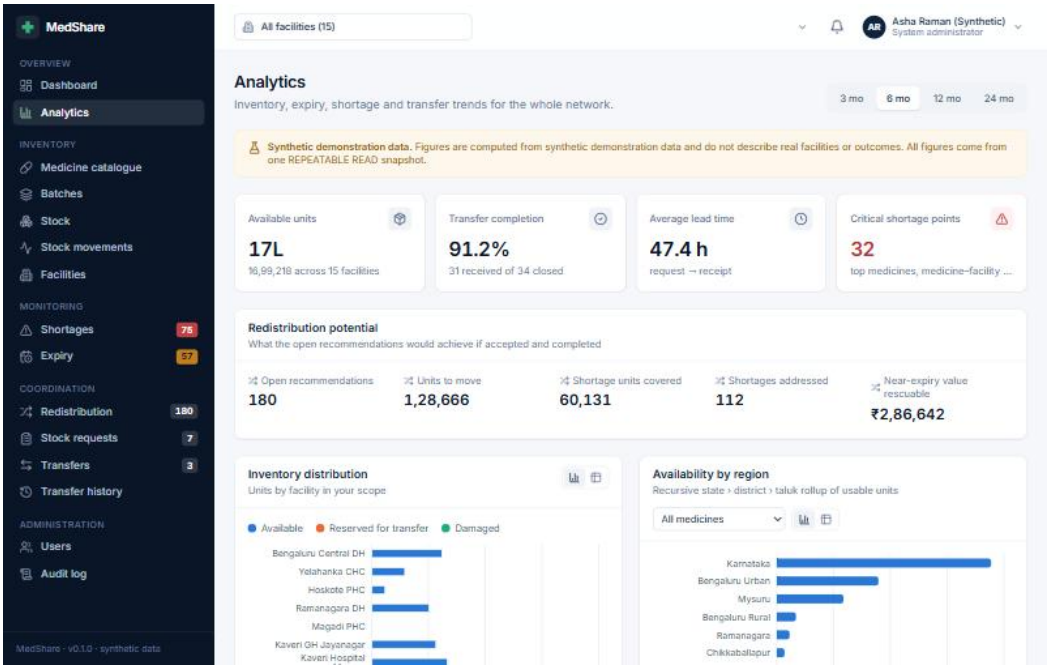
A received transfer: approvals, items (8 units damaged in transit) and timeline



Shortage monitoring with the demand basis of each row



Redistribution recommendations with explanations and priority



Analytics for the whole network, labelled as synthetic data

10. Testing and verification

Suite	Count	What it proves
SQL integrity checks	77	Ledger, transfer lifecycle, contention and rollback, authorisation, business rules, idempotency, alerts, engine, reconciliation
Concurrency checks	22	Row-lock blocking, exactly-N winners, single dispatch, idempotent races, deadlock retry, SERIALIZABLE abort, least privilege
API integration tests	55	Auth, permissions and scope, catalogue, stock operations, idempotent retries, transfer workflow, monitoring, analytics
Frontend tests	19	Permissions, login, transfer actions from the server, receipt with damage, date formatting
SQL query catalogue	24	Every documented query runs read-only as the application role

10.1 Browser verification (end to end)

- **fac-002.inventory** requested 1,608 units of Omeprazole from FAC-001 starting from the shortage page.
- **fac-001.manager** saw it under "Needs my action" and approved it: FAC-001 went from 2,910 available / 0 reserved to 1,302 / 1,608. Then dispatched it.
- **fac-002.inventory** received it with 8 units damaged: the ledger shows TRANSFER_IN +1,600 and TRANSIT_DAMAGE +8, and the shortage dropped from Critical to Low.
- A user asking for another facility's stock gets 403 "Access restricted"; all 22 pages fit a 375 px phone screen without sideways scrolling.

10.2 Bugs found and fixed in Phase 5

Bug	Fix
Dispense, damage, dispose, write-off and adjust ignored the Idempotency-Key, so a retried request could record a movement twice.	Migration 008 adds an exactly-once wrapper; 2 regression tests.
"Needs my action" listed people's own requests and missed organisation-stage approvals.	The filter now matches the server's allowed actions; 2 regression tests.
Read-only auditors could acknowledge alerts.	Acknowledging now requires inventory:write; 1 regression test.

11. Performance study

The four key queries were measured with EXPLAIN (ANALYZE, BUFFERS) on a scaled synthetic database: 2.0 million ledger movements, 224,000 inventory rows, 100,000 transfers, 9,050 medicines and 300 facilities. Each candidate change was applied inside a transaction, measured and rolled back, so "before" and "after" ran on identical data.

Query	Before	After	Decision
Medicine search (full page request)	7.9 s	0.37 s	Migration 009 adopted (22× faster)
Finding source stock for a transfer (FEFO)	589 ms	43 ms	Migration 009 + query rewrite adopted (14× faster)
Monthly consumption (job / analytics read)	2.1 s / 89 ms	unchanged	Already served by indexes and the materialized view
Pending transfers queue	4 ms	unchanged	Already served by an existing composite index

Finding: no new index was justified. The slow queries came from a date function marked VOLATILE (evaluated about 1.3 million times and blocking query inlining), a subquery executed 9,050 times, and distance/permission functions computed per row instead of per facility. Migration 009 fixed these, and a fingerprint of every row of the affected views proved the results identical before and after.

12. Project files and documentation

Path	Contents
README.md	Overview, installation, environment, screenshots, testing
DBMS_DOCUMENTATION.md	Full DBMS write-up in 14 sections, including the 24 SQL queries
ER_DIAGRAM.md	Entity-relationship diagram (crow's foot)
API_CONTRACT.md	Every endpoint, request, response, error code and permission rule
docs/DEMO_WALKTHROUGH.md	15-minute scripted university demonstration
backend/db/migrations/	The schema as SQL migrations 000–009
backend/db/seed/, db/tests/, db/queries/, db/perf/	Synthetic seed, SQL tests, query catalogue, performance data
backend/src/	Express API (auth, modules, error mapping)
backend/tests/	API integration tests
backend/docs/	Phase 1 design, Phase 2 implementation, Phase 5 testing and optimisation, EXPLAIN results
frontend/src/	React app (api layer, auth, components, feature pages)
frontend/docs/	Frontend architecture, UI components, API integration
.env	Local secrets (database passwords, JWT secret); git-ignored

13. Limitations

- Distances are straight-line (haversine), not road distance.
- Demand is a 30-day average with a fallback estimate, not a forecast.
- The recommendation engine is greedy, not a global optimiser.
- The medicine catalogue page aggregates network stock for every medicine before paging (≈ 0.37 s at 9,000 medicines); a pre-computed summary would be faster but less fresh.
- MedShare is a simulation using synthetic data and is not validated for clinical or regulatory use.